



A deep learning framework for derivation of time-stable difference schemes

Nek Sharan^{*1}, Mahesh Natarajan^{†2}, Peter T. Brady^{‡1}, and Daniel Livescu.^{§1}

¹*Los Alamos National Laboratory, Los Alamos, NM, 87544*

²*National Renewable Energy Laboratory, Golden, CO 80401*

High-order centered finite-difference schemes are commonly used for accurate turbulent flow simulations because of their non-dissipative property and computational efficiency. At boundary (grid) points, centered schemes are not applicable, and hence biased stencils or numerical ghost-point treatments are often applied to impose the boundary conditions. For discrete conservation and global high-order accuracy, biased stencils are preferred over the ghost-point treatments. Biased stencils require a careful choice of coefficients for long-time stability, particularly in calculations using centered interior schemes because they do not contain numerical dissipation that may damp the instabilities arising at the boundaries. Stable boundary stencil derivation requires coefficients that satisfy a set of non-linear constraints which are not always analytically tractable. To simplify the derivation, a controllable generation approach using deep neural networks is explored in this study, where the stable stencil coefficients are mapped from a random distribution in two steps. In the first step, the weights and biases of a neural network (NN) are determined to minimize a cost function comprised of the unstable eigenvalues of the Lyapunov operator for the system matrix. In the second step, the NN weights and biases are fixed and the input noise is modified (using gradients from the backward pass of the NN) to obtain stable stencil coefficients. The efficacy of the proposed approach is demonstrated by deriving time-stable boundary stencils for first-derivative approximations with fourth- and sixth-order centered schemes in the interior.

I. Introduction

High-fidelity turbulent flow [1–4] and acoustic wave propagation [5–8] simulations are typically performed with centered finite-difference schemes to avoid erroneous (numerical) damping of flow fluctuations [9, 10]. Long-time calculations in these applications require stable boundary closures that ensure discrete conservation with uniform high-order accuracy. Various numerical stability definitions exist that impose different solution bounds [11]. In this study, we derive boundary stencils that satisfy the time stability (also called strict or energy stability) definition [12, 13], which provides a uniform solution bound in time, preventing non-physical temporal growth.

To precisely define time stability and to determine the numerical conditions on boundary stencil coefficients to ensure time stability, consider the scalar hyperbolic equation

$$\frac{\partial U}{\partial t} + \frac{\partial U}{\partial x} = 0, \quad 0 \leq x \leq 1, \quad t \geq 0, \quad (1)$$

^{*}CCS-2, Los Alamos National Laboratory. AIAA Member.

[†]National Renewable Energy Laboratory.

[‡]CCS-2, Los Alamos National Laboratory. AIAA Member.

[§]CCS-2, Los Alamos National Laboratory. AIAA Associate Fellow.

with the initial and the boundary condition given by

$$U(x, 0) = f(x), \quad U(0, t) = g(t). \quad (2)$$

On a one-dimensional domain with $n + 1$ equidistant grid points $(x_0 = 0, x_1, \dots, x_{n-1}, x_n = 1)$, a semi-discretization of (1)–(2) using strong (or exact) boundary conditions can be written as (e.g. [14])

$$\frac{d\tilde{\mathbf{u}}}{dt} = -D\mathbf{u}, \quad (3)$$

$$\mathbf{u}(0) = \mathbf{f}, \quad (4)$$

where $\mathbf{u}(t) = [u_0(t) \ \dots \ u_n(t)]^T$, with $u_0(t) \equiv g(t)$, is the discrete solution vector. $\tilde{\mathbf{u}}(t) = [u_1(t) \ \dots \ u_n(t)]^T$ is the solution vector without the first element, where the boundary data is directly injected. D , a matrix of size $n \times (n + 1)$, denotes the derivative operator. The entries of D are denoted by d_{ij} , where $1 \leq i \leq n$ and $0 \leq j \leq n$. Its non-square structure prevents computation at the first grid point, where physical boundary condition is applied. $\mathbf{f} = [f(x_0) \ \dots \ f(x_n)]^T$ denotes the discrete initial data.

Define a scalar product and norm for discrete real-valued vector functions $\mathbf{v} = [v_1 \ \dots \ v_n]^T$ and $\mathbf{w} = [w_1 \ \dots \ w_n]^T$ by (e.g. [15])

$$(\mathbf{v}, \mathbf{w})_H = \mathbf{v}^T H \mathbf{w} = \sum_{i,j=1}^{\kappa} h_{ij} v_i w_j \Delta x + \sum_{i=\kappa+1}^{n-\kappa} v_i w_i \Delta x + \sum_{i,j=n-\kappa+1}^n h_{ij} v_i w_j \Delta x, \quad (5)$$

$$\|\mathbf{v}\|_H = \sqrt{(\mathbf{v}, \mathbf{v})_H}, \quad (6)$$

where Δx denotes the grid spacing, κ represents the depth of the boundary stencil, and $h_{i,j}$ are the coefficients of a symmetric positive-definite (norm) matrix H .

Multiplying (3) by $\tilde{\mathbf{u}}^T H$, where H is a norm matrix of size $n \times n$, and adding its transpose yields

$$\frac{d}{dt} \|\tilde{\mathbf{u}}\|_H^2 = -\tilde{\mathbf{u}}^T H D \mathbf{u} - (D \mathbf{u})^T H \tilde{\mathbf{u}}. \quad (7)$$

Using Definition 2.13 of [11], time-stability is defined as:

Definition 1. The approximation (3) is time-stable if for $g = 0$, there is a unique solution $\tilde{\mathbf{u}}(t)$ satisfying

$$\|\tilde{\mathbf{u}}\|_H \leq K \|\tilde{\mathbf{f}}\|_H, \quad \text{or} \quad \frac{d}{dt} \|\tilde{\mathbf{u}}\|_H^2 \leq 0, \quad (8)$$

where K is independent of Δx , $\mathbf{f}$ and t . $\tilde{\mathbf{f}}$ denotes the vector $\mathbf{f}$ without its first element, following the definition of $\tilde{\mathbf{u}}$.

For $g = 0$, the first element of vector $\mathbf{u}$ is zero, i.e. $u_0 = 0$. Substituting $u_0 = 0$ in (7) yields

$$\frac{d}{dt} \|\tilde{\mathbf{u}}\|_H^2 = -\tilde{\mathbf{u}}^T H \tilde{D} \tilde{\mathbf{u}} - (\tilde{D} \tilde{\mathbf{u}})^T H \tilde{\mathbf{u}} = \tilde{\mathbf{u}}^T [H M + (H M)^T] \tilde{\mathbf{u}}, \quad (9)$$

where $M = -\tilde{D}$ and $\tilde{D}$ is a square $(n \times n)$ matrix containing all columns of D except the first. If the approximation (3) is time-stable, i.e. (8) is true, then the following result about the eigenvalues of M can be stated.

Theorem 1. If there exists a positive definite matrix H such that the Lyapunov operator $H M + M^T H$ is negative definite (semi-definite), then the real part of all eigenvalues of M are negative (non-positive).

Proof. See [16, Lemma 3.1.1]. $\square$

Theorem 1 provides a time-stability condition for the boundary stencils, *i.e.* if the stencil coefficients ensure that all eigenvalues of the system matrix M have negative real part then the semi-discretization (3) is time stable. In practice, however, determination of matrix coefficients such that they ensure positive/negative definiteness is challenging [17], especially when the number of free parameters become large in high-order operators [18, 19]. To address this issue, numerical optimization [20, 21] as well as theoretical [22, 23] approaches have been proposed for uniform and cut-cell grids. These approaches have been successful in deriving high-order boundary stencils, however, they are analytically cumbersome or entail very high computational costs, especially for high-order cut-cell stencil derivation. In this study, we propose a neural network (NN) based framework that combines aspects of theoretical and numerical optimization analyses to simplify the derivation of boundary stencils for non-dissipative centered interior schemes.

The paper is organized as follows. Section II describes the controllable generation procedure proposed in this study for derivation of time-stable boundary stencils. Section III provides a third- and a fourth-order time-stable scheme derived using the proposed framework. Numerical results from the application of the derived boundary stencils are discussed in section IV, and the conclusions are provided in section V.

II. NN-based framework for time-stable boundary stencils

Deep generative models [24, 25] have had considerable success in image synthesis in recent years. They typically learn the non-linear mapping from a latent distribution to the real data, and given an input noise (also called the latent code), they can generate high-quality images of a certain class, *e.g.* people, birds, dogs, scenery etc. This class-based image synthesis requires labeled training data and is referred to as conditional generation. Recent works (*e.g.* [26, 27]) have also shown that by carefully editing the input noise, semantic information, *e.g.* the age, gender, hair color etc. of facial images, can be individually targeted and modified. These procedures, referred to as controllable generation, only change the target attribute in an image while maintaining the other image features. In this study, we use the controllable generation concepts to modify the values of the boundary stencil coefficients for time stability.

The goal is to derive the entries of the derivative approximation D and the norm matrix H such that the time-stability definition (Definition 1) is satisfied by the semi-discretization (3) while ensuring high-order accuracy and discrete conservation. To make the discussion concrete, consider a general form of the operators D and H given by

$$D = \frac{1}{\Delta x} \begin{bmatrix} d_{10} & \cdots & \cdots & \cdots & \cdots & d_{1\beta} \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ d_{\kappa 0} & \cdots & \cdots & \cdots & \cdots & d_{\kappa \beta} \\ & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ & & -c_w & \cdots & -c_1 & 0 & c_1 & \cdots & c_w \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \end{bmatrix}, \quad (10)$$

$$H = \Delta x \begin{bmatrix} h_{11} & \cdots & \cdots & h_{1\kappa} \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ h_{\kappa 1} & \cdots & \cdots & h_{\kappa\kappa} \\ & & & & 1 \\ & & & & & \ddots \end{bmatrix}, \quad (11)$$

where κ is the boundary stencil depth and β defines the maximum width of the boundary derivative stencil. c_i are the coefficients of the centered derivative approximation in the interior, which for a half-stencil width of w are given by

$$c_i = -\frac{(-1)^i (w!)^2}{i (w+i)! (w-i)!} \quad \text{for} \quad 1 \leq i \leq w. \quad (12)$$

More details on the structure of these operators for time stability and conservation with strong boundary conditions (BCs) can be found in [14]. In (10) and (11), stencil coefficients for only the inflow boundary are shown because they are the unknowns of interest here. The coefficients for outflow boundary that ensure time stability and conservation with strong BCs are available in literature [14]. While the discussion here uses operators for solving partial differential equations, the proposed approach should be applicable to any problem where the entries of a system matrix need to be determined for stability.

A schematic of the overall procedure is shown in figure 1. The unknowns d_{ij} ($1 \leq i \leq \kappa$, $0 \leq j \leq \beta$) and h_{ij} ($1 \leq i, j \leq \kappa$) in (10) and (11), respectively, are reduced by imposing the accuracy and conservation conditions that provide linear constraints (see *e.g.* [14]). β and κ are chosen accordingly to allow sufficient number of free parameters. The remaining free parameters, after satisfying the accuracy and conservation constraints, are denoted by $\mathbf{p} \in \mathbb{R}^k$ and are determined from the generator. The generator can be regarded as a deterministic function $g : \mathcal{Z} \rightarrow \mathcal{P}$ that maps the m -dimensional latent space $\mathcal{Z}$ (from which the input noise vector $\mathbf{z} \in \mathbb{R}^m$ is sampled) to the k -dimensional free parameter space $\mathcal{P}$, where each sample $\mathbf{p}$ provides the remaining free parameters to construct the operators D and H . From D and H , the symmetric matrix $\mathcal{L} = HM + M^T H$ is constructed, see (9). For time stability, $\mathcal{L}$ must be negative semi-definite and H must be positive definite. Both $\mathcal{L}$ and H are real symmetric matrices, and hence their eigenvalues are real. Using the positive (unstable) eigenvalues of $\mathcal{L}$ and the negative eigenvalues of H , the cost function is defined as

$$J = \text{loss}(\Lambda_{\mathcal{L}}^+, \sigma_{\mathcal{L}} \mathbf{I}_{\mathcal{L}}^+) + \text{loss}(\Lambda_H^-, \sigma_H \mathbf{I}_H^-) = \sum_{i=0}^{n-1} \left(\Lambda_{\mathcal{L},i}^+ - \sigma_{\mathcal{L}} I_{\mathcal{L},i}^+ \right)^2 + \sum_{i=0}^{n-1} \left(\Lambda_{H,i}^- - \sigma_H I_{H,i}^- \right)^2, \quad (13)$$

where $\Lambda_{\mathcal{L}}^+ = \frac{\Lambda_{\mathcal{L}} + |\Lambda_{\mathcal{L}}|}{2} = \text{diag} \left(\Lambda_{\mathcal{L},0}^+ \quad \cdots \quad \Lambda_{\mathcal{L},n-1}^+ \right)$ with $\Lambda_{\mathcal{L}}$ denoting a diagonal matrix containing the eigenvalues of $\mathcal{L}$ and $\Lambda_H^- = \frac{\Lambda_H - |\Lambda_H|}{2} = \text{diag} \left(\Lambda_{H,0}^- \quad \cdots \quad \Lambda_{H,n-1}^- \right)$ with Λ_H denoting a diagonal matrix containing the eigenvalues of H . $\mathbf{I}_{\mathcal{L}}^+$ is a diagonal matrix with entries $I_{\mathcal{L},i}^+ = 1$ if $\Lambda_{\mathcal{L},i}^+$ is non-zero and $I_{\mathcal{L},i}^+ = 0$ otherwise. Similarly, $\mathbf{I}_H^-$ is a diagonal matrix with entries $I_{H,i}^- = 1$ if $\Lambda_{H,i}^-$ is non-zero and $I_{H,i}^- = 0$ otherwise. A small negative value is chosen for $\sigma_{\mathcal{L}}$ and a small positive value is chosen for σ_H to drive the eigenvalues towards time stability. Here, $\sigma_{\mathcal{L}} = -0.2$ and $\sigma_H = 0.2$. n denotes the size of the matrices H and $\mathcal{L}$, as discussed in the previous section.

The steps in NN training and input noise vector manipulation employed here are slightly different than the conventional approach of NN training that requires a lot of data. The output of the NN here (see figure 1) are the boundary stencil free parameters that make the semi-discretization (3) time stable. Numerous values of these free

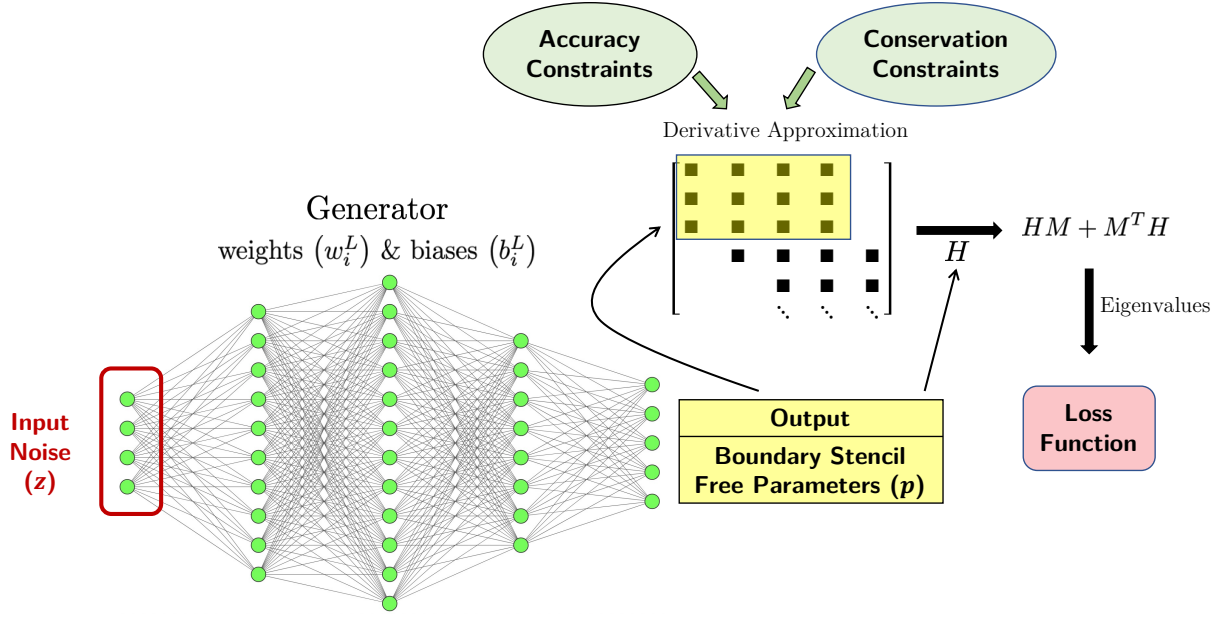


Fig. 1 Schematic of the controllable generation procedure for derivation of time-stable boundary stencils.

parameters (that ensure time stability) are not available to serve as data for NN training, and hence the purpose of this study. In the absence of training data, we use Theorem 1 to score the NN output via the cost function (13). The free parameter values using this procedure are obtained in two steps.

In the first step, the input noise vector $\mathbf{z}$ is sampled from a normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{I}_m)$, where $\mathbf{I}_m$ denotes an identity matrix of size m , and the weights and biases of the hidden layers as well as the noise vector are updated in the backward pass. The noise vector is updated using

$$\mathbf{z}_{\text{new}} = \mathbf{z} + \alpha \frac{\partial J}{\partial \mathbf{z}}, \quad (14)$$

where $\alpha \in [0.01, 0.05]$ is chosen. Around 100 iterations (forward and backward passes) are carried out for each sampled $\mathbf{z}$, and fresh vectors are sampled until a $J \approx O(10)$ is attained. The hidden layer weights and biases are then fixed and $\mathbf{z}$ is further optimized in the second step.

In the second step, the hidden layer parameters are frozen at values corresponding to a cost function of $O(10)$ and the associated $\mathbf{z}$ is used as input to further optimize its value with a smaller learning rate in (14). Values of $\alpha \in [0.0005, 0.01]$ were used in the second step for the stencils reported in the next section. The two steps can be described as a coarse-grained search followed by a fine tuning of the input noise vector to minimize the cost function.

The framework discussed above and depicted in figure 1 was implemented in PyTorch, where `torch.symeig` was used for eigenvalue calculations and `nn.MSELoss(reduction='sum')` was used to calculate the mean squared error in the cost function. The optimization is performed with the Adam algorithm [28] using `torch.optim.Adam` with a learning rate of 0.01. The size of the input and the output layers and the number of hidden layers and nodes for the two schemes derived in this study are specified in the next section.

III. Time-stable strong BC stencils for high-order schemes

Using the framework described in the previous section, boundary stencils of order-of-accuracy q_b are derived for centered interior schemes of order-of-accuracy $q_i = 4$ and 6. If the method is time stable, the global order-of-accuracy of such a scheme, where $q_b < q_i$, is expected to be $q_b + 1$ for first-order hyperbolic systems [29, 30].

A. Third-order scheme

Operators with $q_b = 2$ and $q_i = 4$ are considered for the global third-order accuracy. The details of the NN: the input vector size (m), the number of hidden layers, the number of nodes in each hidden layer, and the output layer size (k), are provided in table 1.

The operators D and H , as defined in section I, for this scheme are given by

$$D = \frac{1}{\Delta x} \begin{bmatrix} d_{10} & d_{11} & d_{12} & d_{13} & d_{14} & d_{15} & d_{16} \\ d_{20} & d_{21} & d_{22} & d_{23} & d_{24} & d_{25} & d_{26} \\ d_{30} & d_{31} & d_{32} & d_{33} & d_{34} & d_{35} & d_{36} \\ d_{40} & d_{41} & d_{42} & d_{43} & d_{44} & d_{45} & d_{46} \\ & & \frac{1}{12} & -\frac{2}{3} & 0 & \frac{2}{3} & -\frac{1}{12} \\ & & & \ddots & \ddots & \ddots & \ddots \\ & & & & \frac{1}{12} & -\frac{2}{3} & 0 & \frac{2}{3} & -\frac{1}{12} \\ & & & & & \frac{4}{49} & -\frac{32}{49} & 0 & \frac{59}{98} & 0 & -\frac{3}{98} \\ & & & & & 0 & \frac{4}{43} & -\frac{59}{86} & 0 & \frac{59}{86} & -\frac{4}{43} \\ & & & & & 0 & 0 & 0 & -\frac{1}{2} & 0 & \frac{1}{2} \\ & & & & & 0 & 0 & \frac{3}{34} & \frac{4}{17} & -\frac{59}{34} & \frac{24}{17} \end{bmatrix}, \quad (15)$$

$$H = \Delta x \text{diag} \left(h_{11}, h_{22}, h_{33}, h_{44}, 1, \dots, 1, \frac{49}{48}, \frac{43}{48}, \frac{59}{48}, \frac{17}{48} \right), \quad (16)$$

where

$h_{11} = 1.493146419525146$	$h_{22} = 1.065617561340332$	$h_{33} = 0.905251026153564$	$h_{44} = 1.035984992980957$
$d_{10} = -0.547026540522789$	$d_{20} = 0.033299806946389$	$d_{30} = 0.031210745222756$	$d_{40} = 0.025372547302312$
$d_{11} = 0.123250345366209$	$d_{21} = -0.577867576569591$	$d_{31} = 0.010657429695129$	$d_{41} = -0.025697013088361$
$d_{12} = 0.412408207038108$	$d_{22} = 0.033803888030439$	$d_{32} = -0.627181263188831$	$d_{42} = 0.022662027270126$
$d_{13} = -0.006461288083685$	$d_{23} = 0.532795725862339$	$d_{33} = 0.005491759168158$	$d_{43} = -0.587090730667114$
$d_{14} = 0.017829276202158$	$d_{24} = -0.022031844269576$	$d_{34} = 0.671876826335880$	$d_{44} = 0.001681949729518$
$d_{15} = 0$	$d_{25} = 0$	$d_{35} = -0.092055497233092$	$d_{45} = 0.643509965089737$
$d_{16} = 0$	$d_{26} = 0$	$d_{36} = 0$	$d_{46} = -0.080438745636217$

The empty spaces in the operator denote zero entry.

Scheme	q_b	q_i	norm	NN details			
				number of hidden layers	m	k	nodes in hidden layers
Third order	2	4	diagonal	3	6	5	[22, 33, 11]
Fourth order	3	6	full	3	8	7	[22, 33, 11]

Table 1 Summary of the properties of the two schemes and the NN architecture used to obtain the schemes.

B. Fourth-order scheme

Operators with $q_b = 3$ and $q_i = 6$ are considered for the global fourth-order accuracy. The details of the NN are provided in table 1. In contrast to a diagonal norm matrix H used for the third-order scheme, a full norm is used here. The operators for this scheme are given by

$$D = \frac{1}{\Delta x} \begin{bmatrix} d_{10} & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & d_{17} \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ d_{40} & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & d_{47} \\ & & -\frac{1}{60} & \frac{3}{20} & -\frac{3}{4} & 0 & \frac{3}{4} & -\frac{3}{20} & \frac{1}{60} \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ & & & & -\frac{1}{60} & \frac{3}{20} & -\frac{3}{4} & 0 & \frac{3}{4} & -\frac{3}{20} & \frac{1}{60} \\ & & & & & d_{n-3,n-6} & \cdots & \cdots & \cdots & \cdots & \cdots & d_{n-3,n} \\ & & & & & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ & & & & & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ & & & & & d_{n,n-6} & \cdots & \cdots & \cdots & \cdots & \cdots & d_{n,n} \end{bmatrix}. \quad (17)$$

$$H = \Delta x \begin{bmatrix} h_{11} & \cdots & \cdots & h_{14} \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ h_{41} & \cdots & \cdots & h_{44} \\ & & & 1 \\ & & & \ddots \\ & & & 1 \\ & & & & h_{n-3,n-3} & \cdots & \cdots & h_{n-3,n} \\ & & & & \vdots & \ddots & \ddots & \vdots \\ & & & & \vdots & \ddots & \ddots & \vdots \\ & & & & h_{n,n-3} & \cdots & \cdots & h_{n,n} \end{bmatrix}, \quad (18)$$

where the boundary stencil coefficients are as given in Appendix A.

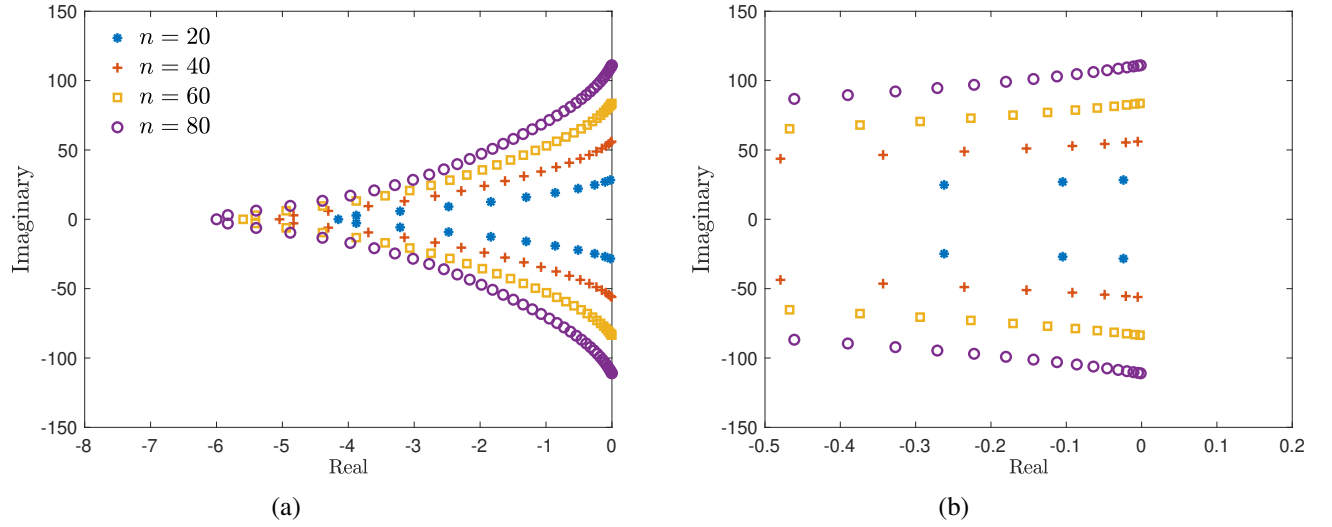


Fig. 2 Eigenvalue spectrum of the system matrix to solve (1) with initial and boundary condition given by (19) using the third-order scheme for various number of grid points. (a) All eigenvalues, (b) Magnified view near the imaginary axis. Legend is the same for both plots.

IV. Numerical results

In this section, numerical results from application of boundary stencils derived in the previous section are discussed. Time integration is performed using the classical fourth-order Runge-Kutta (RK4) method. For convergence studies, the time step is taken small enough such that the temporal errors are insignificant compared to the spatial truncation errors.

Consider the scalar hyperbolic equation (1), on a spatial domain $0 \leq x \leq 1$, with initial and boundary conditions given by

$$u(x, 0) = \sin 2\pi x, \quad u(0, t) = g(t) = \sin 2\pi (-t). \quad (19)$$

The exact solution to the problem is $u(x, t) = \sin 2\pi (x - t)$. A semi-discretization to the problem using strong BCs with the notation of (3) is given by

$$\frac{d\tilde{\mathbf{u}}}{dt} = -D\mathbf{u} = -\tilde{D}\tilde{\mathbf{u}} - \mathbf{d}_0 g, \quad (20)$$

where $\mathbf{d}_0$ denotes the first column of the derivative operator D . For a bounded boundary data $g(t)$, the stability of the semi-discretization depends on the properties of the matrix $M = -\tilde{D}$, referred to as the system matrix.

Figures 2 and 3 show the eigenvalue spectrum of the system matrix using various grid points for the third- and the fourth-order schemes, respectively. All eigenvalues lie in strict left half of the complex plane and, therefore, the discretization is time-stable.

Table 2 shows the L_2 - and L_∞ -norm of the solution error, denoted by ε , and the respective convergence rates from the two schemes. Both schemes exhibit the expected order of accuracy.

V. Conclusions

A controllable generation approach using deep neural networks is proposed for derivation of time-stable high-order finite-difference schemes that impose boundary conditions strongly (or exactly). The proposed approach is general in

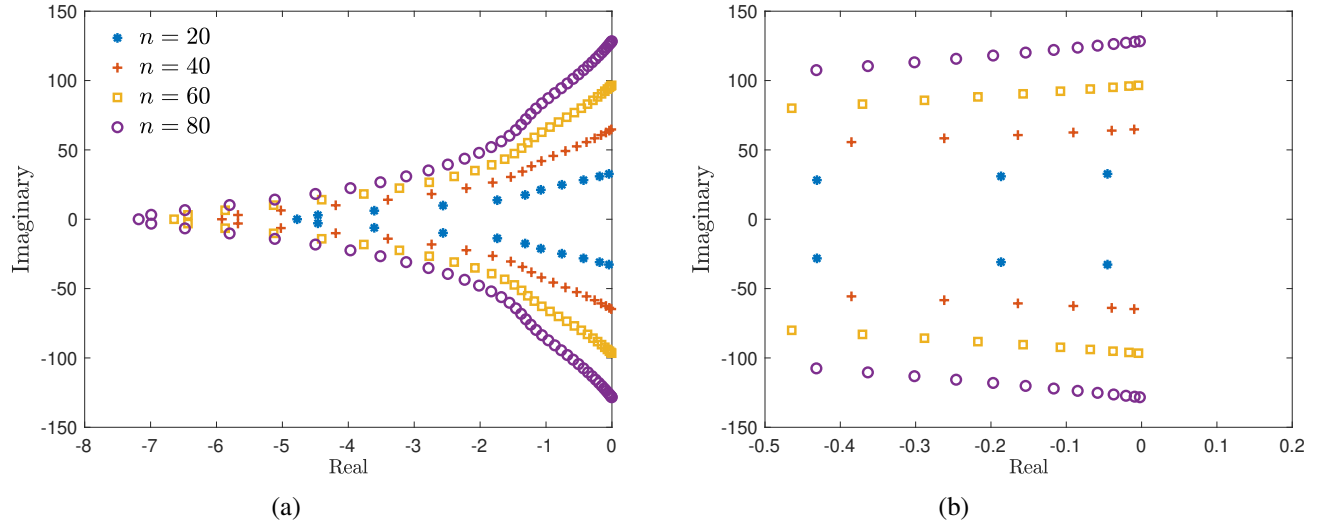


Fig. 3 Eigenvalue spectrum of the system matrix to solve (1) with initial and boundary condition given by (19) using the fourth-order scheme for various number of grid points. (a) All eigenvalues, (b) Magnified view near the imaginary axis. Legend is the same for both plots.

n	Third-order scheme				Fourth-order scheme			
	$\log_{10} \ \varepsilon\ _2$	Rate	$\log_{10} \ \varepsilon\ _\infty$	Rate	$\log_{10} \ \varepsilon\ _2$	Rate	$\log_{10} \ \varepsilon\ _\infty$	Rate
20	-1.973422		-1.703554		-2.279161		-1.948476	
40	-2.916809	3.134	-2.574815	2.894	-3.449783	3.889	-3.196234	4.145
80	-3.835924	3.053	-3.486709	3.029	-4.631867	3.927	-4.366281	3.887
160	-4.745814	3.023	-4.384746	2.983	-5.826623	3.969	-5.557024	3.956
320	-5.651926	3.010	-5.282615	2.983	-7.026841	3.987	-6.756342	3.984
640	-6.556365	3.004	-6.197129	3.038	-8.229245	3.994	-7.958516	3.994

Table 2 L_2 - and L_∞ -norm of the error and convergence rates for the two schemes. Error calculations performed at $t_f = 1.0$.

that it can be applied to determine system matrix coefficients for stability in any application. The generation procedure does not require training data, instead the node weights and biases of the neural network are trained using a cost function based on the eigenvalues of the output. A global third- and fourth-order scheme is derived using the proposed approach and their stability and accuracy are evaluated for an inviscid scalar hyperbolic problem.

Acknowledgements

This work was supported by the US Department of Energy through the Los Alamos National Laboratory. Los Alamos National Laboratory is operated by Triad National Security, LLC, for the National Nuclear Security Administration of U.S. Department of Energy (Contract No. 89233218CNA000001). Research presented in this article was supported by the Laboratory Directed Research and Development program of Los Alamos National Laboratory under project number 20190227ER.

A. Boundary stencils for the fourth-order scheme

The entries of the derivative operator (17) are given by

$$\begin{aligned}
& \begin{aligned} d_{10} &= -0.539535565299040 & d_{20} &= 0.062659099765849 & d_{30} &= 0.120196559905155 & d_{40} &= 0.045992741665226 \\ d_{11} &= 0.048576403522867 & d_{21} &= -0.363153776811457 & d_{31} &= -0.227434168737852 & d_{41} &= -0.165618683609376 \\ d_{12} &= 0.932632912447246 & d_{22} &= -1.056815408616417 & d_{32} &= -0.647159168692265 & d_{42} &= 0.278149718708140 \\ d_{13} &= -1.274989378627789 & d_{23} &= 2.285853786421574 & d_{33} &= 0.632320752322092 & d_{43} &= -0.690065967905702 \\ d_{14} &= 1.352511771714427 & d_{24} &= -1.338587348431032 & d_{34} &= -0.049220877622433 & d_{44} &= -0.151938934887303 \\ d_{15} &= -0.632327349412170 & d_{25} &= 0.500258970016849 & d_{35} &= 0.212714578916925 & d_{45} &= 0.833148104185622 \\ d_{16} &= 0.129202871912064 & d_{26} &= -0.103785034837013 & d_{36} &= -0.048024597702566 & d_{46} &= -0.169201215627550 \\ d_{17} &= -0.016071666257604 & d_{27} &= 0.013569712491647 & d_{37} &= 0.006606921610944 & d_{47} &= 0.019534237470944 \end{aligned} \\
& \begin{aligned} d_{n-3,n-6} &= -0.016561684177261 & d_{n-2,n-6} &= -0.000361638454691 & d_{n-1,n-6} &= 0.000926155340965 & d_{n,n-6} &= -0.001445257338767 \\ d_{n-3,n-5} &= 0.148693519140655 & d_{n-2,n-5} &= -0.013405297064079 & d_{n-1,n-5} &= -0.008973007982595 & d_{n,n-5} &= 0.010557093852612 \\ d_{n-3,n-4} &= -0.741094886543546 & d_{n-2,n-4} &= 0.133029048031689 & d_{n-1,n-4} &= 0.030656091288366 & d_{n,n-4} &= -0.032019044358739 \\ d_{n-3,n-3} &= -0.024655295020487 & d_{n-2,n-3} &= -0.724163785725475 & d_{n-1,n-3} &= 0.115249274519848 & d_{n,n-3} &= -0.281922947649156 \\ d_{n-3,n-2} &= 0.782025275575096 & d_{n-2,n-2} &= 0.013794616447447 & d_{n-1,n-2} &= -0.953846621578270 & d_{n,n-2} &= 1.453991030655289 \\ d_{n-3,n-1} &= -0.169802610554259 & d_{n-2,n-1} &= 0.681982248203315 & d_{n-1,n-1} &= 0.478629162310713 & d_{n,n-1} &= -2.978250966158610 \\ d_{n-3,n} &= 0.021395681579801 & d_{n-2,n} &= -0.090875191438207 & d_{n-1,n} &= 0.337358946100972 & d_{n,n} &= 1.829090090997372 \end{aligned}
\end{aligned}$$

The entries of the norm matrix (18) are given by

$$\begin{aligned}
& \begin{aligned} h_{11} &= 0.678009152412415 & h_{21} &= 0.602291312169731 & h_{31} &= -0.162859448871374 & h_{41} &= 0.194520850520015 \\ h_{12} &= 0.602291312169731 & h_{22} &= 1.209105491638184 & h_{32} &= -1.151989288045012 & h_{42} &= 0.045239236469778 \\ h_{13} &= -0.162859448871374 & h_{23} &= -1.151989288045012 & h_{33} &= 3.355408430099487 & h_{43} &= -0.468622323288211 \\ h_{14} &= 0.194520850520015 & h_{24} &= 0.045239236469778 & h_{34} &= -0.468622323288211 & h_{44} &= 1.140316247940063 \end{aligned}
\end{aligned}$$

$$\begin{array}{llll}
h_{n-3,n-3} = 1.010565909795035 & h_{n-2,n-3} = -0.022159678607914 & h_{n-1,n-3} = 0.049290321977698 & h_{n,n-3} = -0.011307664275930 \\
h_{n-3,n-2} = -0.022159678607914 & h_{n-2,n-2} = 1.009529522678526 & h_{n-1,n-2} = -0.066752758640902 & h_{n,n-2} = -0.041450418763043 \\
h_{n-3,n-1} = 0.049290321977698 & h_{n-2,n-1} = -0.066752758640902 & h_{n-1,n-1} = 1.103973967122970 & h_{n,n-1} = 0.159321802873567 \\
h_{n-3,n} = -0.011307664275930 & h_{n-2,n} = -0.041450418763043 & h_{n-1,n} = 0.159321802873567 & h_{n,n} = 0.242047391276517
\end{array}$$

References

- [1] Rai, M. M., and Moin, P., “Direct simulations of turbulent flow using finite-difference schemes,” *Journal of Computational Physics*, Vol. 96, No. 1, 1991, pp. 15–53.
- [2] Morinishi, Y., Lund, T. S., Vasilyev, O. V., and Moin, P., “Fully conservative higher order finite difference schemes for incompressible flow,” *Journal of Computational Physics*, Vol. 143, No. 1, 1998, pp. 90–124.
- [3] Ryu, J., and Livescu, D., “Turbulence structure behind the shock in canonical shock–vortical turbulence interaction,” *Journal of Fluid Mechanics*, Vol. 756, 2014.
- [4] Sharan, N., Matheou, G., and Dimotakis, P. E., “Turbulent shear-layer mixing: initial conditions, and direct-numerical and large-eddy simulations,” *Journal of Fluid Mechanics*, Vol. 877, 2019, pp. 35–81.
- [5] Tam, C. K., and Dong, Z., “Wall boundary conditions for high-order finite-difference schemes in computational aeroacoustics,” *Theoretical and Computational Fluid Dynamics*, Vol. 6, No. 5-6, 1994, pp. 303–322.
- [6] Colonius, T., and Lele, S. K., “Computational aeroacoustics: progress on nonlinear problems of sound generation,” *Progress in Aerospace Sciences*, Vol. 40, No. 6, 2004, pp. 345–416.
- [7] Saurabh, S., “Direct numerical simulation of human phonation,” Ph.D. thesis, University of Illinois at Urbana-Champaign, 2017.
- [8] Natarajan, M., Freund, J. B., and Bodony, D. J., “Global mode-based control of laminar and turbulent high-speed jets,” *Comptes Rendus Mécanique*, Vol. 346, No. 10, 2018, pp. 978–996.
- [9] Mittal, R., and Moin, P., “Suitability of upwind-biased finite difference schemes for large-eddy simulation of turbulent flows,” *AIAA journal*, Vol. 35, No. 8, 1997, pp. 1415–1417.
- [10] Sharan, N., Matheou, G., and Dimotakis, P. E., “Mixing, scalar boundedness, and numerical dissipation in large-eddy simulations,” *Journal of Computational Physics*, Vol. 369, 2018, pp. 148–172.
- [11] Gustafsson, B., *High order difference methods for time dependent PDE*, Vol. 38, Springer Science & Business Media, 2007.
- [12] Trefethen, L. N., “Stability of finite-difference models containing two boundaries or interfaces,” *Mathematics of Computation*, Vol. 45, No. 172, 1985, pp. 279–300.
- [13] Carpenter, M. H., Gottlieb, D., and Abarbanel, S., “Time-stable boundary conditions for finite-difference schemes solving hyperbolic systems: methodology and application to high-order compact schemes,” *Journal of Computational Physics*, Vol. 111, No. 2, 1994, pp. 220–236.
- [14] Sharan, N., Brady, P. T., and Livescu, D., “Stable and conservative boundary treatment for difference methods, with application to cut-cell discretizations,” *AIAA Scitech 2020 Forum*, 2020, p. 0807.
- [15] Kreiss, H.-O., and Scherer, G., “On the existence of energy estimates for difference approximations for hyperbolic systems,” Tech. rep., Technical report, Dept. of Scientific Computing, Uppsala University, 1977.

- [16] Corless, M. J., and Frazho, A., *Linear systems and control: an operator perspective*, CRC Press, 2003.
- [17] Van Loan, C. F., and Golub, G., "Matrix computations (Johns Hopkins studies in mathematical sciences)," 1996.
- [18] Sharan, N., "Time-stable high-order finite difference methods for overset grids," Ph.D. thesis, University of Illinois at Urbana-Champaign, 2016.
- [19] Sharan, N., Pantano, C., and Bodony, D. J., "Time-stable overset grid method for hyperbolic problems using summation-by-parts operators," *Journal of Computational Physics*, Vol. 361, 2018, pp. 199–230.
- [20] Brady, P. T., and Livescu, D., "High-order, stable, and conservative boundary schemes for central and compact finite differences," *Computers & Fluids*, Vol. 183, 2019, pp. 84–101.
- [21] Brady, P., and Livescu, D., "Foundations for high-order, conservative cut-cell methods: Stable discretizations on degenerate meshes," *Journal of Computational Physics*, 2020, p. 109794.
- [22] Sharan, N., Pantano, C., and Bodony, D. J., "Energy stable overset grid methods for hyperbolic problems," *7th AIAA Theoretical Fluid Mechanics Conference*, 2014, p. 2924.
- [23] Sharan, N., Brady, P. T., and Livescu, D., "Finite-difference Cartesian cut-cell method for hyperbolic systems," *AIAA Scitech 2021 Forum*, 2021, p. 0746.
- [24] Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y., "Generative adversarial networks," *Advances in neural information processing systems*, Vol. 3, No. 06, 2014.
- [25] Salakhutdinov, R., "Learning deep generative models," *Annual Review of Statistics and Its Application*, Vol. 2, 2015, pp. 361–385.
- [26] Shen, Y., Gu, J., Tang, X., and Zhou, B., "Interpreting the latent space of gans for semantic face editing," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 9243–9252.
- [27] Yang, C., Shen, Y., and Zhou, B., "Semantic hierarchy emerges in deep generative representations for scene synthesis," *International Journal of Computer Vision*, Vol. 129, No. 5, 2021, pp. 1451–1466.
- [28] Kingma, D. P., and Ba, J., "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [29] Gustafsson, B., "The convergence rate for difference approximations to mixed initial boundary value problems," *Mathematics of Computation*, Vol. 29, No. 130, 1975, pp. 396–406.
- [30] Gustafsson, B., "The convergence rate for difference approximations to general mixed initial-boundary value problems," *SIAM Journal on Numerical Analysis*, Vol. 18, No. 2, 1981, pp. 179–190.